Sub-goal 1: Simple Folding

In this coursework, one of our group members applied for a medical leave of absence for the academic year, resulting in only three members in our group. Hence, while we were exempted from the Part 1 task as advised by Professor Loc, since some of the functions in Part 1 are indispensable for the subsequent parts, we have still decided to complete the development of Part 1. However, in the report, we will only briefly introduce its functions and will not provide detailed information.
In Part 1, we implemented the Constant Folding, including:
Numeric conversions of constants: `iconst_3; i2l → lconst_3`
Arithmetic on two constants: `iconst_2; iconst_5; iadd` → `bipush 7`
These functions have provided assistance for our subsequent optimization.

Sub-goal 2: Constant Variables

Approach
The main objective of the Constant Variable Propagation optimization is to replace loads of local numeric variables with their known constant values when it is safe to do so. This reduces unnecessary reads of local variables and, more importantly, creates new opportunities for constant folding to further simplify expressions.
The implementation is intentionally conservative: we target only numeric types (int/long/float/double) and explicitly exclude local variables of reference types (ALOAD/ASTORE) to prevent error propagation. A local variable slot is eligible for propagation only if all of the following hold:

- It's numeric locals only, and can't be a reference type
- In the whole method, it should be exactly one write to the slot

This ensures correctness without needing to build a CFG or compute reaching definitions.
Algorithm
The algorithm for Sub-goal 2 is implemented as a single method (propagateConstantVariables(...)) that performs two scans over the method's InstructionList.
Phase 1 — Identify constant local variables
In the first phase, we filter the variables that are loaded only once.
The pass iterates through the InstructionList and maintains:

- storeCount: counts how many times each local index is written.
- constantVars: records a candidate constant value for a local index.

For each instruction, the pass first checks whether it is a numeric StoreInstruction and not an ASTORE. If so, it extracts the destination local-variable slot index via getIndex() and increments the corresponding write count in storeCount. This write count is later used to conservatively restrict propagation to locals that are written exactly once.

The pass then inspects the store's immediately preceding instruction (prev). If prev exists, it calls getTypedConstant(prev, inst, cpgen) to determine whether the value being stored is produced by a type-compatible constant push directly before the store. If a constant can be extracted, the local slot is recorded in constantVars as a candidate. Otherwise, the slot is removed from constantVars, conservatively invalidating it because the pass cannot prove that the assigned value is a constant under the restricted pattern it supports. If the store has no preceding instruction (prev == null), the slot is also removed from constantVars for the same reason.

In addition to explicit stores, the pass treats IINC as a write to the corresponding local slot because it modifies the variable's value. Therefore, when an IINC instruction is encountered, the pass increments storeCount for that index and removes the index from constantVars, ensuring that locals affected by IINC are not considered constant candidates.

After the iteration is complete, we double-check each candidate's store count. If the store count is larger than 1, we remove it as well.

Phase 2 — Replace loads of these variables with their constant values
After phase 1 is complete, we scan the instruction list for numeric LoadInstructions and, for any load whose local slot is recorded in constantVars, replace the load with an equivalent typed constant-push instruction. The replacement is performed in place via h.setInstruction(...), and the method returns whether the instruction was changed, allowing the outer optimisation loop to repeat passes as new constant-folding opportunities arise.

Example Optimisation Outcome
In the constant folder example, the original instruction is:

```
  public int methodOne();
    descriptor: ()I
    flags: (0x0001) ACC_PUBLIC
    Code:
      stack=2, locals=3, args_size=1
         0: bipush        62
         2: istore_1
         3: iload_1
         4: sipush        764
         7: iadd
         8: iconst_3
         9: imul
        10: istore_2
        11: iload_2
        12: sipush        1234
        15: iadd
        16: iload_1
        17: isub
        18: ireturn
```

After the optimisation, the instruction becomes:

```
public int methodOne();
   descriptor: ()I
   flags: (0x0001) ACC_PUBLIC
   Code:
     stack=1, locals=1, args_size=1
        0: sipush        3650
        3: ireturn
```

This shows that, with the help of part 1 and dead code elimination, our system can optimise the value that is used only once.

Sub goal 3: Dynamic Variables

Approach

The main objective of Dynamic Variable propagation optimisation is to replace loads of local numerical values with their known constant values, but for specific intervals: starting from the assignment (or initialisation) until the next assignment. This optimisation targets variables that are reassigned with different constant values during the scope of a method. Unlike constant variables, which maintain a single value throughout, dynamic variables may hold different constant values at different points in the program. By tracking these value intervals, the optimiser can replace variable loads with the corresponding constant value for each interval for further constant folding of arithmetic expressions that use these variables. This reduces the number of variable loads and arithmetic operations executed at runtime, as expressions involving known constants can be evaluated at compile time.

Algorithm

Similar to sub-goal 2, the algorithm for Sub-goal 3 is implemented as a single method `propagateDynamicVariables()` that performs two scans over the method's InstructionList.

Scan 1: Identify variables modified by IINC

In Java bytecode, the IINC instruction modifies a local variable directly without using store instructions, as shown in the following example:

Java code	bytecode

`a = b - 67`	`11: iload_2`
	`12: bipush  67`
	`14: isub`
	`15: istore_1`

i++	43: iinc 3, 1

Unlike normal assignment, IINC does not produce a StoreInstruction, so it would not be detected during the main propagation scan. If these variables are not excluded from the following optimization method, the optimizer may incorrectly replace their loads with a stale constant value.

To prevent this, the first scan iterates through all instructions, identifies every variable modified by IINC, and records their slot indices in a HashMap. These variables are then excluded from constant propagation throughout the second scan.

Scan 2: Propagate constant values, replace variable loads and fold arithmetic
The second scan iterates through the instruction list linearly, maintaining a HashMap that maps each variable's slot index to its current known constant value. The scan handles three cases:

Case 1: StoreInstruction
When a StoreInstruction is encountered, the algorithm checks whether the preceding instruction pushes a constant value. If so, the variable's entry in the HashMap is updated with the new constant. If the value being stored is not a constant, or if the variable was identified as an IINC variable in Scan 1, it is removed from the HashMap to prevent incorrect propagation.

Case 2:
When a LoadInstruction is encountered, the algorithm checks if the loaded variable has a known constant value in the HashMap. If so, the load instruction is replaced with a constant push instruction, making the final executable program directly loading the value from the HashMap.

Case 3:
When an ArithmeticInstruction is encountered, the algorithm checks whether the two preceding instructions are both constant push instructions. If so, the arithmetic operation is evaluated at compile time, the two preceding push instructions are deleted, and the arithmetic instruction is replaced with a single push instruction containing the computed result. When deleting instructions, a `TargetLostException` may occur if the deleted instructions are branch targets, in which case the targeters are redirected to the replacement instruction.

Example Optimisation Outcome
The original and optimized bytecode of DynamicVariableFolding `methodFour()` is listed here.

Original:
```
public int methodFour();
    descriptor: ()I
    flags: (0x0001) ACC_PUBLIC
    Code:
      stack=5, locals=4, args_size=1
```

```
         0: ldc           #20                 // int 534245
         2: istore_1
         3: iload_1
         4: sipush        1234
         7: isub
         8: istore_2
         9: getstatic     #8                  // Field
java/lang/System.out:Ljava/io/PrintStream;
        12: ldc           #21                 // int 120298345
        14: iload_1
        15: isub
        16: i2d
        17: ldc2_w        #22                 // double 38.435792873d
        20: dmul
        21: invokevirtual #24                 // Method
java/io/PrintStream.println:(D)V
        24: iconst_0
        25: istore_3
        26: iload_3
    27: bipush        10
        29: if_icmpge     49
        32: getstatic     #8                  // Field
java/lang/System.out:Ljava/io/PrintStream;
        35: iload_2
        36: iload_1
        37: isub
        38: iload_3
        39: imul
        40: invokevirtual #27                 // Method
java/io/PrintStream.println:(I)V
        43: iinc          3, 1
        46: goto          26
        49: iconst_4
        50: istore_1
        51: iload_1
        52: iconst_2
        53: iadd
        54: istore_2
        55: iload_1
        56: iload_2
        57: imul
        58: ireturn
```

Optimized:

```
public int methodFour();
```

```
    descriptor: ()I
    flags: (0x0001) ACC_PUBLIC
    Code:
      stack=3, locals=4, args_size=1
         0: getstatic     #8                  // Field
java/lang/System.out:Ljava/io/PrintStream;
         3: ldc2_w        #44                 // double
4.603228141221259E9d
         6: invokevirtual #24                 // Method
java/io/PrintStream.println:(D)V
         9: iconst_0
        10: istore_3
        11: iload_3
        12: bipush        10
        14: if_icmpge     34
        17: getstatic     #8                  // Field
java/lang/System.out:Ljava/io/PrintStream;
        20: sipush        -1234
        23: iload_3
        24: imul
        25: invokevirtual #27                 // Method
java/io/PrintStream.println:(I)V
        28: iinc          3, 1
        31: goto          11
        34: iconst_4
        35: istore_1
        36: bipush        24
        38: ireturn
```

Sub-goal 4: Additional Peephole Optimisation

Additional Peephole Optimisation Choice
: Dead Store Removal

Approach
The main objective of the Dead Store Removal additional optimisation is to eliminate redundant local variable stores that remain even after simplifications of unnecessary expressions and constants through earlier optimisations (sub-goals 1, 2, 3). We particularly focused on addressing the two common patterns that cause dead stores: 1) Where a store's value is overwritten before any read occurs, and 2) Where a store to a local variable is never read at all due to previous optimisations done to achieve sub-goals 1, 2, 3.

The implementation is kept conservative, solely considering the numeric local variable instructions and excluding reference stores such as ‘ASTORE’, because object references introduce aliasing and concerns regarding side-effects that are outside the main scope of this coursework. It also avoids directly eliminating instructions at control-flow boundaries, such as branch targets, because it is difficult for the local linear scan to guarantee accurate dead store elimination when multiple incoming paths are involved. Lastly, the additional optimisation approach removes a store only when the instruction immediately before it is a push instruction that produces a single constant. By restricting it in such a way, the implementation ensures a simple and safe deletion of both the useless value producer and the useless store without requiring one to reconstruct a stack.

Algorithm

The algorithm developed for the sub-goal 4 deploys two separate passes, both called from the main optimisation loop in the ‘ConstantFolder’ class inside ConstantFolder.java.

Firstly, ‘removeOverwrittenStores(...)’ removes stores that are overwritten by a subsequent store to the same local before any reads in between. The algorithm functions as follows:

1. Iterates through the method’s ‘InstructionList’
2. For each instruction in the ‘InstructionList’, checks whether it is a numeric ‘StoreInstruction’ rather than ‘ASTORE’
3. Ignores the store if it is a branch target to prevent the risk of invalidating control-flow joins
4. Records the local variable index being written
5. Scans forward each instruction from the following bytecode position
6. Stops scanning if a branch target is reached, or if the current instruction is a ‘BranchInstruction’ or ‘Select’
7. If a load of the same local variable is found before any redefinition, the original store is not considered a dead store, so the algorithm keeps the store
8. If the subsequent ‘StoreInstruction’ to the same index is found before any load, the original or earlier store is considered to have been overwritten and redundant
9. If an IINC on the same index is found before any load, the earlier stores are considered to have been overwritten as IINC changes the variable’s value before reading the stored value
10. If the store is detected as redundant, the algorithm checks the preceding instruction to delete the store, only if that preceding instruction is a single constant push and not a branch target
11. Removes both instructions together

Secondly, ‘removeDeadStores(...)’ removes numeric locals that are never read or loaded subsequently. Its algorithm works as follows:

1. Does a first scan over the method to count all numeric ‘LoadInstruction’s using local variable index

2. Counts IINC as a read-like use of the local because incrementing a variable involves reading the current value and updating it
3. Does a second scan over the method to look for numeric ‘StoreInstruction’s
4. If the local variable has no recorded uses, the corresponding store is considered dead
5. Likewise, only deletes the store if the store is not a branch target, where the preceding instruction exists, is not a branch target, and is a single constant push
6. Removes the constant push and store

Both ‘removeOverwrittenStores(...)’ and ‘removeDeadStores(...)’ return a boolean of whether they changed the instruction list. Hence, if a dead store removal operation leads to a new potential optimisation, the outer loop can apply them repeatedly in later rounds of further optimisations as well.

Implementation

The relevant methods to achieve the sub-goal 4 are the following:

- ‘optimize()’ calls ‘removeOverwrittenStores(...)’ and ‘removeDeadStores(...)’ in task 4
- ‘removeOverwrittenStores(...)’ removes stores that are overwritten by subsequent stores before the stored local variables are used in between
- ‘removeDeadStores(...)’ removes stores that are never read subsequently to locals
- ‘isBranchTarget(...)’ is a helper to prevent unsafe deletions at control-flow points across both methods (‘removeOverwrittenStores(...)’ and ‘removeDeadStores(...)’)
- ‘isSinglePushConstant(...)’ is a helper to recognise instructions that produce a single constant, which can be safely deleted with a dead store across both methods (‘removeOverwrittenStores(...)’ and ‘removeDeadStores(...)’)
- ‘retargetLostTargets(...)’ is reused when instruction removal done in task 4 causes BCEL to report lost targets

A detailed description of the implementation is noted below.

The dead store optimisation starts from the ‘optimize()’ method. For each method in the class, it createes a ‘MethodGen’ and acquires its ‘InstructionList’. Then, it repeatedly applies several optimisation passes inside a do…while loop. The optimisation, specifically written for the sub-goal 4 is invoked near the end of each round of the thorough optimisation through ‘removeOverwrittenStores(...)’ and ‘removeDeadStores(...)’, so that dead store elimination step is done only after running earlier folding and propagation tasks (for sub-goals 1, 2, 3) to remove unnecessary or unused expressions or constants after the preliminary optimisation process.

Specifically, the ‘removeOverwrittenStores(...)’ iterates over the list of instructions (using the ‘InstructionHandle’ pointer), stores the current handle in ‘cur’ and advances ‘h’ before identifying possible stores to delete. This was done to allow the traversal to continue safely even after the current instruction pair is deleted, as the next pointer is saved first.

The first filter checks 1) ‘inst instanceof StoreInstruction’ and ‘!(inst instanceof ASTORE)’, indicating that the pass only analyses stores containing primitive or numeric local variables (e.g. ‘ISTORE’, ‘LSTORE’, ‘FSTORE’, ‘DSTORE’).

The second filter (‘isBranchTarget(cur)’ inspects the instruction’s targeters and returns true if jump or switch cases land on the instruction (i.e. if any targeter is a ‘BranchInstruction’ or ‘Select’, for the rationale outlined under the Algorithm sub-section).

If the store passes the two filters, the code extracts the local variable slot (using ‘int idx = ((StoreInstruction) inst).getIndex();), where each slot index indicates where each local variable is being written.

The method scans forward from ‘cur.getNext()’, searching for the next event that involves the same local variable. There are four key cases at this stage:

1. ‘isBranchTarget(scan)’ : If the scan reaches an instruction that is a branch target, the search stops, because the linear format between the original store and subsequent instruction is no longer supported nor guaranteed
2. ‘scanInstruction instanceof BranchInstruction || scanInstruction instanceof Select’ : Similarly, if a branch or switch instruction is encountered, the scan stops
3. Same index of a numeric load : If the ‘LoadInstruction’ has the same ‘idx’ as previously-processed, the value written by the original store has been read before being overwritten. If this is the case, the store is not dead, and hence, must not be removed.
4. Later store or ‘IINC’ of the same local variable : If another ‘StoreInstruction’ writes the same ‘idx’, or an ‘IINC’ modifies the same local variable, it implies that the original value in the store is being replaced before being read. Hence, the earlier store is considered redundant.

The ‘overwritten’ flag records whether the fourth case occurred before the third case. If not, the pass continues to the next candidate store without considering it redundant. Even when a store is considered redundant, the implementation still does not delete it immediately; instead, it looks at the preceding instruction (‘InstructionHandle prev = cur.getPrev();’) and verifies that it satisfies the ‘isSinglePushConstant(...)’. Simply put, this means that Task 4 only removes store patterns of the constant push and store to local.
This addresses cases where the constant is pushed and immediately consumed by the store by removing both instructions to preserve stack optimisation.

Deleting is done through ‘il.delete(prev, cur);’. In case BCEL throws a ‘TargetLostException’ when deleted instructions are still referenced, the methods catch the exception and retarget any lost references to a nearby valid instruction through ‘retargetLostTargets(...)’, where the replacement is chosen using ‘cur.getNext()’ if available, and ‘prev.getPrev()’ otherwise.

The ‘removeDeadStores(...)’ used in Task 4 is structured differently because it detects variables that are never used anywhere in the method, rather than stores overwritten along a linear path.

The method begins with a first pass over the list of instructions by building a ‘HashMap<Integer, Integer>’ called ‘loads’. Its key is the index of the local variable, and the value is the number of observed uses of the local variable. The use is counted for every numeric ‘LoadInstruction’ or an ‘IINC’ instruction found by the method.

After counting the total uses, the method makes a second pass over all instructions to look for numeric stores. For each ‘StoreInstruction’, it checks whether the count is non-zero (‘loads.getOrDefault(idx, 0) != 0’). If so, it implies that the local variable is used somewhere, so the store is not globally dead, whereas if the count is zero, then no numeric load or increment reads the variable in the method, thereby considering it dead.

Similar to the ‘removeOverwrittenStores(...)’ method, safety checks ensure that the store is not a branch target and the preceding instruction exists, is not a branch target, and is a single constant push, before deleting the ‘prev, cur’ pair.

Example Optimisation Outcome
In the ‘original.txt’ file’s ‘*methodFour()*’, the original bytecode was as follows:

```
0: ldc2_w     #5      // long 48357834231
3: lstore_1
4: ldc2_w     #7      // long 400000l
7: lstore_3
8: lload_1
9: lload_3
10: ladd
11: lstore       5
13: lload_1
14: lload_3
15: lcmp
16: ifle          23
…
```

As shown, the result of the addition operation is stored in local variable 5, but the local variable is never used or loaded afterwards. ‘methodFour()’ only uses ‘lload_1’ and ‘lload_3’ again for the comparison, without loading local variable 5 that stores the sum value.
Hence, after optimisation, ‘*methodFour()*’ in the ‘optimised.txt’ file became:

```
0: ldc2_w        #5     // long 48357834231
3: ldc2_w        #7     // long 400000l
6: lcmp
7: ifle               14
10: iconst_1
11: goto           15
14: iconst_0
15: ireturn
```

The optimisation done for Task 4 has successfully removed a dead store, an unused temporary value of local variable 5.

Additional Peephole Optimisation Choice (one additional optional optimisation done): Constant Branch Folding and Unreachable Code Removal

Part 4-B focuses on simplifying bytecode control flow when branch outcomes can be determined at compile time, and then removing code that becomes unreachable after control-flow simplification. The work consists of two cooperating optimisations. The first is constant branch folding, which evaluates comparisons or conditional branches with constant operands and rewrites them into simpler instructions, typically unconditional jumps. The second is unreachable code removal, which deletes instruction sequences that can no longer be executed after terminators such as GOTO, return instructions, or ATHROW.

Algorithm

Sub-aim 1 — Folding constant comparisons and branches

The method iterates to a fixed point using a do-while loop. Each iteration scans the instruction list from the beginning. Once a transformation is applied, the pass breaks out immediately and restarts scanning. This design avoids subtle issues from deleting instructions during traversal and ensures that newly exposed optimisation opportunities are discovered in subsequent iterations.

The first set of transformations folds comparison instructions when both operands are constants directly preceding the comparison. For LCMP, the pass examines the two previous instruction handles and attempts to extract two long constants using getPushedLongConstant. If both are present, it computes the comparison result using Long.compare, which yields -1, 0, or 1. It then replaces the first constant-push instruction with a new typed PUSH of that integer result and deletes the second constant push together with the LCMP instruction. This reduces a three-instruction sequence to a single constant push, often enabling additional folding of subsequent conditional branches.

For floating-point comparisons, the pass handles FCMPL/FCMPG and DCMPL/DCMPG in the same structural way, but with correct JVM NaN behaviour. It extracts the two constants using getPushedFloatConstant or getPushedDoubleConstant. If either operand is NaN, the result is forced to -1 for the “L” variant (FCMPL/DCMPL) and 1 for the “G” variant (FCMPG/DCMPG). Otherwise, it uses Float.compare or Double.compare. As with LCMP, it replaces the first push with a PUSH of the computed integer result and deletes the second push and the compare instruction.

After comparison folding, the method folds constant-conditional branches for non-Select-branch instructions. For unary integer branches, it checks the immediately preceding instruction and attempts to extract an int constant via getPushedIntConstant. If successful, it evaluates whether the branch for that constant would be taken. It then deletes the constant-push instruction (since the conditional no longer needs a stack operand) and replaces the conditional branch with an unconditional GOTO. If the condition is true, the new GOTO targets the original branch target. If the condition is false, it targets the fall-through instruction, defined as h.getNext(). This rewrite preserves behaviour while removing the runtime test.

For binary integer branches, the pass inspects the two immediately preceding instruction handles, extracts two int constants, evaluates the comparison to determine whether the branch

is taken, deletes both constant pushes, and replaces the conditional branch with a GOTO to either the original target or the fall-through successor.
In addition to folding conditionals, the method performs a small control-flow cleanup by removing redundant unconditional jumps. If a GOTO targets exactly the next instruction (the fall-through), then the GOTO has no effect. The code deletes such a GOTO provided the instruction handle itself is not a branch target, because deleting a target instruction could invalidate existing jumps that point to it.
All deletions are wrapped in try/catch blocks for TargetLostException, which can occur when a deleted instruction is still referenced as a branch target. When this happens, the pass calls retargetLostTargets to redirect lost targets to a safe surviving instruction handle, typically the handle that remains after replacement. This ensures the bytecode remains well-formed after transformations.
The method returns a boolean indicating whether any change was made. This allows the outer optimisation driver to rerun passes, and within the method, the fixed-point loop ensures the transformations are applied exhaustively.

Sub-aim 2 — Removing unreachable code

After folding is complete, the unreachable-code pass scans the instruction list and looks for terminator instructions that end the current control-flow path. Terminators include GOTO, all return opcodes, and ATHROW. Once a terminator is encountered, any subsequent instruction is unreachable unless some other instruction branches to it. Therefore, starting from the instruction immediately after the terminator, the pass deletes consecutive instructions until it reaches either the end of the list or an instruction that is a branch target (as detected by isBranchTarget). Encountering a branch target stops deletion, because that instruction may be reachable via a jump from elsewhere.
As with the folding pass, deletions may raise TargetLostException. When this occurs, the pass retargets lost references to a safe instruction handle, preferring the next surviving instruction if available; otherwise, it retargets to the terminator itself. This keeps all jump targets valid while still removing unreachable sequences.

Example Optimisation Outcome

In the constant folder example, the original instruction is:

```
 public boolean methodFour();
   descriptor: ()Z
   flags: (0x0001) ACC_PUBLIC
   Code:
        stack=4, locals=7, args_size=1
           0: ldc2_w        #10                 // long 48357834231
           3: lstore_1
           4: ldc2_w        #12                 // long 4000001
           7: lstore_3
           8: lload_1
           9: lload_3
          10: ladd
          11: lstore        5
          13: lload_1
```

```
        14: lload_3
        15: lcmp
        16: ifle          23
        19: iconst_1
        20: goto          24
        23: iconst_0
        24: ireturn
      StackMapTable: number_of_entries = 2
        frame_type = 254 /* append */
          offset_delta = 23
          locals = [ long, long, long ]
        frame_type = 64 /* same_locals_1_stack_item */
          stack = [ int ]
}
```

After optimisation, the result becomes:

```
  public boolean methodFour();
    descriptor: ()Z
    flags: (0x0001) ACC_PUBLIC
    Code:
      stack=1, locals=1, args_size=1
         0: iconst_1
         1: ireturn
}
```

Platform/OS

All experiments were implemented in Java and executed on both macOS and UCL's department Linux machine (Red Hat Enterprise Linux). On macOS, Java 8 was used; on the Linux machine, OpenJDK 17.0.18 LTS was used. Development and testing were carried out using Visual Studio Code as the primary editor.

Contributions

Xinyi Wu: Implemented sub-goal 3 (Dynamic Variables) and wrote the corresponding part in the report.
Jiyu Kim: Implemented sub-goal 4 (Additional Peephole Optimisation) and completed the corresponding section of the report.
Zizhou Shi: Implemented sub-goal 1, 2, and added the foldConstantBranches function for sub-goal 2. Wrote the corresponding part in the report.